Python 기초 문법

자료형 · 제어문 · 함수 · 예외처리

cchyun@gmail.com

Part 1. 자료형 + 변수

로봇 개발 맥락에서의 Python 주요 자료형

- **로봇 시스템에서 데이터는 물리적인 수치나 상태를 나타냄**
  - `int` **(정수):** 이산적인 상태 표현
    - 엔코더 틱 수, 디지털 센서 상태 코드(0 또는 1), 모터 단계 수
  - `float` **(실수):** 연속적인 물리량 표현
    - 로봇 좌표(x, y, z), 이동 속도(m/s), 배터리 전압(V), 장애물 거리
  - `str` **(문자열):** 텍스트 기반 정보·식별자
    - 로봇 이름(`"BumperBot"`), 상태 메시지(`"Emergency Stop Active"`), 포트 이름(`"/dev/ttyUSB0"`)
  - `bool` **(불리언):** 논리적 참/거짓 판단
    - 범퍼 센서 충돌 여부, 배터리 저전압 경고, 시스템 전원 ON/OFF

로봇 개발 맥락에서의 Python 주요 자료형 (2)

- **컬렉션 자료형**
  - `list` **(리스트):** 순서가 있는 데이터 묶음 — 수정 가능
    - Lidar 센서 거리 값 목록, 과거 위치 기록(경로) 저장
  - `tuple` **(튜플):** 리스트와 유사하나 한 번 정해지면 변경 불가
    - 로봇 하드웨어 고정 크기(너비, 높이), 변경 불가능한 초기 설정값
  - `dict` **(딕셔너리):** 키(Key)-값(Value) 쌍으로 빠른 접근
    - 센서 메타데이터: `{"id": 1, "type": "Ultrasonic", "unit": "cm"}`
  - `set` **(집합):** 중복을 허용하지 않는 고유 값들의 모임
    - 현재 시스템에 연결된 고유 센서 ID 목록 관리

동적 타입 언어(Dynamic Typing)란?

- **Python은 동적 타입 언어 — 실행 시점에 값에 따라 타입이 결정됨**
  - **정적 타입(C/Java):** 변수를 만들기 전에 반드시 타입 선언 필요

```
float speed = 0.5;  // 나중에 다른 타입의 값 불가
```

  - **동적 타입(Python):** 타입 선언 없이 값에 따라 자동 인식

```
speed = 0.5        # Python이 float으로 자동 인식
speed = "fast"     # 문법적으로 허용 (주의 필요!)
```

  - **입문자 팁:** 개발 속도는 빠르지만, 대규모 로봇 프로젝트에서는 의도치 않은 타입 오류 발생 가능 → 변수 타입에 주의

변수 명명 규칙과 좋은 이름 짓는 법

- **로봇 소프트웨어는 변수 이름만 보고도 역할을 알 수 있어야 함**

  - **기본 규칙**
    - 영문자, 숫자, 밑줄(`_`)만 사용 / 숫자로 시작 불가
    - 대소문자 구분 (`sensor`와 `Sensor`는 다름)

  - **로봇 개발 스타일 가이드 (PEP 8)**

대상	규칙	예시
변수·함수	`lowercase_with_underscores`	`motor_speed_rpm`, `is_obstacle_detected`
클래스	`UpperCamelCase`	`LidarSensor`, `RobotController`

  - **좋은 변수명 짓기 팁**
    - `v`, `t` 같은 약어 대신 → `linear_velocity`, `elapsed_time`
    - 단위 포함으로 실수 방지 → `distance_cm`, `angle_deg`

자료형 변환(Type Casting) 및 주의사항

- **서로 다른 자료형 간의 연산이 필요할 때 형변환 함수를 사용**

  - **형변환 함수 정리**

함수	변환 방향	예시
`int()`	실수·문자열 → 정수	`int(5.7)` → `5`
`float()`	정수·문자열 → 실수	`float("1.23")` → `1.23`
`str()`	숫자 → 문자열	`str(100)` → `"100"`

  - **로봇 개발 시 주의사항**
    - 센서로부터 넘어온 문자열이 숫자로 변환 불가능한 형식(`"error"`)일 때
    - `float()`를 쓰면 `ValueError`가 발생하여 프로그램이 멈춤
    - **예외 처리(`try-except`)와 함께 사용** 필수

입문자가 자주 하는 실수 — 자료형·변수

- **실수 1: 정수 나눗셈 결과**

```
5 / 2   # 결과: 2.5 (실수)  ← Python 3
5 // 2  # 결과: 2   (정수 몫)  ← // 연산자 사용
```

- **실수 2: 문자열과 숫자 연결**

```
# 오류 발생!
print("거리: " + distance)

# 올바른 방법
print(f"거리: {distance}")        # f-string 사용
print("거리: " + str(distance))  # str() 변환
```

- **실수 3: 리스트 복사 실수**

```
list_b = list_a   # 같은 데이터를 가리킴 → 하나 수정 시 둘 다 변경!
list_b = list_a.copy()  # 올바른 복사 방법
```

예제 코드 — 로봇 센서 데이터 처리 기초

```python
# 1. 변수 선언 및 자료형 확인
robot_name = "Alpha-1"       # str
wheel_radius = 0.05          # float (단위: m)
encoder_count = 1024         # int
is_moving = False            # bool

# 2. 리스트를 이용한 센서 데이터 관리
distance_readings = [120.5, 95.0, 45.2, 10.1, 5.5]  # 최근 5번 측정값 (cm)

# 3. 자료형 변환 및 출력
raw_data = "45.7"
current_distance = float(raw_data)  # 문자열을 실수로 변환

is_dangerous = current_distance < 20.0  # 안전 거리(20cm) 미만 확인

print(f"로봇 이름: {robot_name}")
print(f"현재 측정 거리: {current_distance} cm")
print(f"충돌 위험 여부: {is_dangerous}")

# 4. 자주 하는 실수 방지
# print("Encoder: " + encoder_count)  # 오류 발생!
print("Encoder Count: " + str(encoder_count))  # 올바른 방법
```

Part 2. 제어문 (if / for / while)

if / elif / else 구조와 조건식 작성법

- **if-elif-else 구조: 로봇이 환경에 반응하여 의사결정을 내리는 기본 도구**
  - 안전 임계값(Threshold) 확인, 간단한 상태 머신(State Machine) 구현에 활용
  - `and`, `or`, `not` 논리 연산자로 여러 센서 조건을 동시에 확인 가능

```python
# 초음파 센서 거리 데이터 (단위: 미터)
distance = 0.5
is_emergency_stop_pressed = False

# 장애물 거리 및 비상 정지 버튼 상태 확인
if distance < 0.2 or is_emergency_stop_pressed:
    print("비상 정지! 장애물이 너무 가깝거나 버튼이 눌렸습니다.")
elif distance < 1.0 and distance >= 0.2:
    print("감속 운행: 장애물 감지됨.")
else:
    print("정상 주행: 경로가 확보되었습니다.")
```

for 루프 — 센서 데이터 배열 처리

- **for 문: 시퀀스의 각 항목을 순회 — Lidar 360도 스캔 데이터 처리에 필수**
  - `range()`: 특정 횟수 반복 또는 인덱스 순회
  - `enumerate()`: 인덱스와 값을 동시에 반환
  - `zip()`: 여러 시퀀스를 병렬로 순회

```
# 라이다(Lidar) 센서의 5개 방향 거리 데이터
lidar_readings = [1.2, 0.8, 2.5, 0.4, 3.0]

# enumerate: 방향 인덱스와 거리 데이터를 함께 처리
for i, reading in enumerate(lidar_readings):
    if reading < 0.5:
        print(f"경고: {i}번 방향({reading}m)에 장애물 발견!")

# range: 특정 횟수 제어 명령 송신
for _ in range(3):
    print("회전 신호 송신 중...")
```

while 루프 — 로봇 메인 운영 루프

- **while 루프: 시스템이 켜져 있는 동안 지속되는 로봇의 메인 루프**
  - **"인식-판단-동작(Sense-Think-Act)"** 사이클을 끊임없이 반복
  - `while True:` 무한 루프 + `break` / `continue` 로 흐름 제어

```
battery_level = 100

# 배터리가 20% 이상일 때만 작동하는 제어 루프
while battery_level > 20:
    print(f"로봇 가동 중... 배터리 잔량: {battery_level}%")
    battery_level -= 5

    if battery_level == 50:
        print("중간 점검 중, 다음 사이클로 넘어갑니다.")
        continue  # 아래 코드를 건너뛰고 다음 반복

    if battery_level <= 20:
        print("배터리 부족으로 기지로 복귀합니다.")
        break     # 루프 전체 탈출
```

중첩 루프와 성능 주의사항

- **중첩 루프: 2차원 그리드 맵 스캔, 다중 센서 그룹 처리에 활용**
  - **성능 문제 주의**
    - 고밀도 데이터(예: Lidar 포인트 클라우드)를 일반 파이썬 리스트 + 중첩 루프로 처리 시
    - 계산 속도가 느려져 **제어 주기(Control Cycle)가 깨질 수 있음**
  - **해결 방법: NumPy 벡터화 연산**

```python
import numpy as np

# 중첩 루프 대신 벡터화 연산 (훨씬 빠름)
point_cloud = np.array([[1.2, 0.5], [0.8, 1.1], [2.3, 0.9]])
distances = np.linalg.norm(point_cloud, axis=1)
print(distances)
```

- 실시간 로봇 제어 환경에서는 NumPy 사용이 표준

입문자가 자주 하는 실수 — 제어문

- **실수 1: 들여쓰기 오류 (Indentation Error)**
  - 파이썬은 들여쓰기로 코드 블록을 구분 → **공백 4칸이 표준**

```
for i in range(5):
print("Moving Step", i)  # 오류! 들여쓰기 없음
```

- **실수 2: 무한 루프 실수**
  - `while` 루프의 종료 조건을 갱신하지 않아 프로그램이 멈추지 않음
  - 로봇 제어 시 시스템 자원 과부하로 이어질 수 있음

- **실수 3: 조건식·연산자 혼동**
  - 비교 연산자(`==`)와 할당 연산자(`=`) 혼동
  - 리스트 원본을 수정하며 순회하는 실수

- **실수 4: 하드웨어 예외 처리 미흡**
  - 센서 연결 끊김 등에 대비해 `try-except` 블록과 함께 사용 권장

Part 3. 함수 정의 + 예외처리

함수의 필요성 — 코드 중복 제거 및 재사용성

- **함수: 복잡한 로직을 독립적으로 테스트할 수 있는 모듈식 단위**

  - **코드 중복 제거**
    - PID 제어 계산, 좌표 변환 등 반복 로직을 함수로 캡슐화
    - 필요할 때마다 호출하여 재사용

  - **시스템 복잡성 관리**
    - 단순 스크립트 → 복잡한 플랫폼으로 진화할수록 함수는 필수
    - 상태와 복잡성을 체계적으로 관리

  - **로봇 개발 예시**

```
# 함수 없이: 동일한 PID 계산 코드가 여러 곳에 중복
# 함수 있이: 한 번 정의하고 어디서든 호출
def calculate_pid(error, kp, ki, kd):
    ...
```

함수 정의 문법

- `def` 키워드로 함수 정의 시작
  - **매개변수(Parameters)**: 호출 시 전달된 인자를 함수 내 지역 변수로 저장
  - **기본값(Default Value)**: 인자를 생략해도 기본값으로 동작
  - `return`: 결과값 반환 후 종료 (없으면 `None` 반환)

```python
# 기본 함수 정의
def calculate_distance(speed, time):
    return speed * time

# 기본값이 있는 매개변수
def connect_sensor(port, baudrate=115200, timeout=1.0):
    print(f"포트 {port} 연결 중 (Baudrate: {baudrate})")
    # ...

# 호출 방법
dist = calculate_distance(0.5, 10)
connect_sensor("/dev/ttyUSB0")            # 기본값 사용
connect_sensor("/dev/ttyUSB0", timeout=2) # 특정 인자만 지정
```

가변 인자

- **가변 인자를 유연하게 처리하는 방법**
  - `*args`: 정해지지 않은 수의 **위치 인자** → 튜플 형태로 수신
    - 로봇이 여러 개의 센서 값을 한 번에 처리할 때 유용
  - `**kwargs`: 정해지지 않은 수의 **키워드 인자** → 딕셔너리 형태로 수신
    - 센서 설정값(해상도, 포트 등)을 유연하게 전달할 때 실용적

```python
# *args: 여러 센서 값을 한 번에 처리
def average_readings(*args):
    return sum(args) / len(args)

avg = average_readings(1.2, 0.8, 2.5, 0.4)  # 인자 개수 제한 없음

# **kwargs: 유연한 설정값 전달
def configure_sensor(**kwargs):
    print(f"센서 설정: {kwargs}")

configure_sensor(resolution=0.01, port="/dev/ttyUSB0", mode="continuous")
```

람다(Lambda) 함수와 사용 시점

- `lambda` 키워드: 작은 익명(Anonymous) 함수
  - 특징
    - 단일 표현식으로 제한
    - `def`로 별도 함수 정의 없이 인라인으로 작성
  - 사용 시점
    - `list.sort()`, `filter()`, `map()` 등 함수 객체가 인자로 필요한 곳
    - 짧은 로직을 전달할 때 적합

```
# 람다 함수 사용 예시: 센서 값 필터링
readings = [22.1, 25.4, 105.0, 23.8]  # 105.0은 노이즈로 가정

# 50도 이상의 비정상 데이터 필터링
filtered_data = list(filter(lambda x: x < 50.0, readings))
print(f"필터링된 센서 값: {filtered_data}")

# 리스트 정렬에 람다 활용
sensors = [{"id": 2, "val": 1.5}, {"id": 1, "val": 0.8}]
sensors.sort(key=lambda s: s["id"])  # id 기준 정렬
```

로봇 개발에서 예외 처리의 중요성

- **로봇 하드웨어는 다양한 실패 상황에 노출**
  - 연결 끊김, 센서 응답 시간 초과(Timeout), 통신 포트 충돌 등
  - **예외 처리의 역할**
    - 국지적인 하드웨어 오류가 **시스템 전체 추락(Crash)으로 이어지지 않도록** 방지
    - 오류 발생 시에도 로봇의 **"심박수(Heartbeat)"를 유지**
    - 모든 모터를 정지시키는 등 **안전한 상태(Safe State)** 로 유도
  - **예외 처리 없는 로봇 vs 있는 로봇**

구분	예외 처리 없음	예외 처리 있음
센서 오류 발생	프로그램 즉시 종료	오류 기록 후 계속 동작
하드웨어 복구	수동 재시작 필요	자동 재연결 시도 가능
안전 보장	불가	비상 정지 로직 실행

try / except / else / finally 구조

- **예외 처리의 4가지 블록**
  - `try`: 예외가 발생할 가능성이 있는 코드 배치
  - `except`: 발생한 예외에 대한 대응 로직 (재연결 시도, 데이터 폐기 등)
  - `else`: 예외가 **발생하지 않았을 때만** 실행
  - `finally`: 예외 발생 여부와 **상관없이 항상 실행** — 자원 정리, 안전 위치 이동

```
try:
    data = read_sensor("/dev/ttyUSB0")
except ValueError as e:
    print(f"[경고] 잘못된 데이터: {e}")        # 데이터 문제
except HardwareCommError as e:
    print(f"[위험] 하드웨어 오류: {e}")         # 통신 문제
else:
    print(f"데이터 정상 수신: {data}")           # 성공 시만 실행
finally:
    print("하드웨어 자원 정리 및 안전 모드 확인")  # 항상 실행
```

자주 발생하는 예외 종류

- **로봇 개발에서 자주 마주치는 예외 유형**

예외	발생 상황	로봇 개발 예시
`ValueError`	잘못된 형식의 값	센서 패킷 체크섬 실패
`TypeError`	잘못된 타입의 객체	함수에 float 대신 string 전달
`IndexError`	범위를 벗어난 인덱스	센서 데이터 리스트 범위 초과
`KeyError`	존재하지 않는 키 참조	설정 딕셔너리에 없는 키 접근
`FileNotFoundError`	파일을 찾을 수 없음	설정 파일 또는 로그 파일 없음

- **항상 구체적인 예외부터 처리하고, 마지막에 `Exception`으로 포괄 처리**

```
except ValueError:      # 구체적 예외 먼저
    ...
except Exception as e: # 예상치 못한 예외는 마지막에
    ...
```

커스텀 예외 클래스 만들기

- **표준 예외 외에 로봇 특정 상황을 위한 커스텀 예외 정의 가능**
  - 기본 `Exception` 클래스를 상속받아 구현

```python
import time

# 커스텀 예외 클래스 정의
class HardwareCommError(Exception):
    """하드웨어 통신 관련 사용자 정의 예외"""
    pass

# 함수 정의와 매개변수 활용
def read_sensor_data(sensor_id, timeout=1.0, **kwargs):
    """로봇 센서 데이터를 읽는 함수"""
    print(f"센서 {sensor_id} 읽기 시작 (설정: {kwargs})")

    if sensor_id == "thermal":
        raise ValueError("잘못된 센서 데이터 형식 (Checksum Error)")
    elif sensor_id == "motor_port":
        raise HardwareCommError("모터 컨트롤러 연결 끊김")

    return 25.5  # 정상 데이터 반환
```

예제 코드 — 로봇 센서 및 하드웨어 통신

```python
def main_control_loop():
    sensor_name = "thermal"

    try:
        data = read_sensor_data(sensor_name, frequency=10, mode="raw")

    except ValueError as e:
        # 잘못된 데이터는 버리고 다음 프레임 대기
        print(f"[경고] {e}: 현재 패킷을 버리고 다음 데이터를 기다립니다.")

    except HardwareCommError as e:
        # 치명적 하드웨어 오류 처리
        print(f"[위험] {e}: 비상 정지 시퀀스를 가동합니다.")

    except Exception as e:
        print(f"알 수 없는 오류 발생: {e}")

    else:
        print(f"성공적으로 데이터를 읽었습니다: {data}")

    finally:
        # 오류 여부와 상관없이 항상 실행
        print("하드웨어 자원을 정리하고 안전 모드를 확인합니다.")

if __name__ == "__main__":
    main_control_loop()
```

실습 체크리스트

- **오늘 학습한 내용을 코드로 구현해 보셨나요?**

1. **가상 환경(venv) 설정**
   - 로봇 라이브러리 간의 충돌을 방지하기 위해 프로젝트 전용 가상 환경 생성 및 활성화

2. **데이터 타입 확인**
   - 센서의 소수점 값(`float`)과 상태 코드(`int`)를 적절한 자료형으로 처리

3. **제어 흐름 설계**
   - 로봇의 메인 루프를 `while` 문으로 구현하고, 비상 정지 로직에 `if` 문 사용

4. **예외 처리 적용**
   - 시리얼 통신 끊김이나 잘못된 센서 데이터 입력에 대비해 `try-except` 구문 포함

5. **코드 가독성(PEP 8)**
   - 들여쓰기 4칸 준수, 함수와 변수 이름을 의미 있게 작성